



Energy aware DBSCAN and mobility aware balanced q-learning based opportunistic routing protocol in MANET

A. Daniel Gurwin Das^{1,2} · E. Anna Devi²

Received: 27 January 2025 / Accepted: 29 April 2025 / Published online: 16 May 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

The efficiency of opportunistic Q-learning-based routing protocols in mobile ad hoc networks depends on how well the agent's exploration—exploitation dilemma is managed. This paper focuses on enhancing the decision-making process of a Q-learning-based reinforcement learning agent by optimizing the Candidate Forwarder List (CFL) in an opportunistic routing protocol for Mobile Ad Hoc Networks (MANETs). A single-cluster Density-Based Spatial Clustering of Applications with Noise (DBSCAN) technique is used to generate the First Forwarder List (FFL), which includes nodes with residual energy above 25% and located within 250 meters of the current node. To form the CFL, a mobility-aware threshold distance is computed using the relative speed and distance of neighboring nodes. Nodes that meet this threshold are considered stable and are added to the CFL, reducing link breakage. A modified epsilon-greedy strategy is applied, where the exploration decision is influenced by the mobility threshold, ensuring that only mobility-aware and stable nodes are selected. This balances exploration and exploitation during learning. Simulation results demonstrate that the proposed method achieves a Packet Delivery Ratio (PDR) of 87% and an average end-to-end delay of 0.06 seconds. The exploration-to-exploitation ratio is around 1.033, confirming the effectiveness of the approach in highly dynamic and energy-constrained MANET environments.

Keywords MANET · Q-learning · Opportunistic routing · DBSCAN · Epsilon-greedy · Threshold distance · Cooperative private network

1 Introduction

Unlike reactive or proactive routing protocols, opportunistic routing protocols do not construct or maintain a routing table for packet transmission. Instead, they rely on broadcasting packets to all neighboring nodes until the destination is reached. However, this broadcast-based approach often leads

to challenges such as buffer overflow [1] and unnecessary energy consumption caused by irrelevant packet processing [2].

To address these issues and identify the optimal next-hop neighbor, reinforcement learning—specifically, the Q-learning algorithm—is employed. In this approach, a node initially selects a neighbor and transmits the packet. If an acknowledgment (ACK) is received, the neighbor is marked as a reachable and optimal candidate. Otherwise, the node is considered unreachable, and the algorithm proceeds to evaluate the next available neighbor until a successful acknowledgment is obtained [3].

The decision-making process is guided by a reward mechanism, where successful packet delivery results in a positive reward, and failure to deliver a packet leads to a negative reward. These rewards are critical in enabling the Q-learning agent to learn and make informed routing decisions over time [3]. Sometimes, a node may refuse to forward packets, exhibiting selfish behavior. Such nodes are referred to as *selfish nodes* [4]. Therefore, rewards play a crucial role in the decision-making process of agents.

This article is part of the Topical Collection: *Special Issue on I-Track on Networking and Applications*
Guest Editor: Vojislav B. Misić

✉ A. Daniel Gurwin Das
danielgurwindas@gmail.com

E. Anna Devi
annadevimit@gmail.com

¹ Department of Computing, Kings Cornerstone International College, ECR, Muttukadu, Chennai 603112, Tamil Nadu, India

² Department of Electronics Engineering, Sathyabama Institute of Science and Technology, Jeppiaar Nagar, Chennai 600119, Tamil Nadu, India

Another significant challenge in Mobile Ad Hoc Networks (MANETs) is the mobility of nodes, which leads to frequent and unpredictable topology changes. This dynamic nature complicates the maintenance of accurate neighbor information, as wireless links may break when nodes move out of each other's transmission range [5]. Nodes with high mobility and low residual energy are particularly susceptible to such issues, as they are more likely to experience frequent link breakages and network disconnections.

In such highly dynamic environments, energy-aware routing becomes essential for maintaining network efficiency. Studies have shown that a device's *battery level can be indirectly estimated* by analyzing the *characteristics of received RF signals*, such as signal attenuation patterns and transmission power. These characteristics provide valuable insights into a node's *energy state*, which can be leveraged to make more informed and energy-efficient routing decisions [6, 7].

In addition to energy considerations, the *distance to neighboring nodes* is another key factor that influences routing decisions. It has been demonstrated that a node's proximity to its neighbors can be estimated using the *Received Signal Strength Indicator (RSSI)* values [8, 9]. RSSI-based distance estimation allows for better management of node proximity, which is vital for minimizing link failures and optimizing communication between nodes.

Another critical challenge in MANETs is obtaining accurate mobility information, as nodes are constantly on the move. There are two primary approaches to addressing this: one involves predicting node mobility using machine learning or statistical models [10], while the other relies on the nodes sharing their mobility status directly with their neighbors. The proposed method adopts the latter approach, where each node shares its mobility information with its neighbors. This direct communication helps improve the accuracy of mobility predictions, enabling more informed decision-making by the routing agent.

In this context, the routing agent operates in two states: *exploration* and *exploitation*. During the exploration state, the agent actively searches for the optimal neighbor, while in the exploitation state, it makes decisions based on the Q-table values. However, incorrect neighbor selection during the exploration phase can lead to increased exploration time, higher end-to-end delay, and potential network instability. Additionally, sub-optimal neighbor selection can result in prolonged convergence time [11].

To mitigate these issues, the proposed method carefully considers the convergence challenges that arise from prolonged exploration. To address this, an optimal **Candidate Forwarder List (CFL)** is constructed by excluding nodes with low residual energy and high mobility. This selective process reduces unnecessary exploration, speeds up convergence, and improves overall network performance.

According to the IEEE 802.11ax standard, a 250-meter range is typically considered the maximum distance for classifying a node as a neighbor [12]. Existing methods primarily focus on node connectivity, often overlooking the effects of mobility, suboptimal Candidate Forwarder Lists, and poor agent decision-making during exploration in dynamic environments [2, 13, 14]. To address these limitations, the proposed approach incorporates both distance and mobility information of neighboring nodes to enhance the agent's exploration strategy. This integration effectively reduces exploration time, minimizes end-to-end delay, and improves overall network stability.

Abbreviations

The commonly used abbreviations and notations throughout this manuscript are listed in Table 1 for ease of reference.

Table 1 List of abbreviations used in the manuscript

Abbreviation	Full Form
3D	3-Dimension
Ack	Acknowledgement
c	Choice selector
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DDQN	Dueling Double Q-Network
EDMBOPR	Energy-Aware DBSCAN And Mobility Aware Balanced Q-learning Opportunistic Routing Protocol
ESQAODV	Energy and Signal Strength Q-learning AODV
FFL	First Forwarder List
FL-AMM	Federated Learning Augmented Map Mapping
GPS	Global Positioning System
IFFL	Intermediate First Forwarder List
IoT	Internet of Things
LSTM	Long Short-Term Memory
MANET	Mobile Adhoc Network
MEC	Mobile Edge Computing
m/s	meter per second
m	meter
PDR	Packet Delivery Ratio
P_i	Positional updates of current node i
QSCR	Q-learning-based Smart Clustering Routing Protocol
rc	Random Choice
RORQ	Reputation-based Opportunistic Routing Protocol
RSSI	Received Signal Strength Indicator
T	Threshold Distance
UAV	Unmanned Aerial Vehicles

Table 1 continued

Abbreviation	Full Form
VANET	Vehicular Adhoc Network
d_i	Euclidean distance between two coordinate points of the current node i
d_{ij}	Euclidean distance between the coordinates of the current node i and its neighbor j
ε_i	Exploration rate of current node agent i
v_{ai}	Average speed of the current node i
v_{ij}	Average speed of the neighbor node j as received by the current node i
d	destination
r	reward
ip	IPv4 address
ϵ	Exploration Rate
s	second

2 Related works

A recent work, such as the reputation-based opportunistic routing protocol (RORQ), employs a reinforcement learning technique called Q-learning to identify the most suitable forwarding neighbor. In this protocol, each node maintains a Candidate Forwarder List (CFL) comprising all neighboring nodes within its transmission range. Each node acts as an agent: it selects a forwarding node from the CFL and transmits the packet. If the selected node is not the intended forwarder, the agent receives a negative reward. The agent then selects another neighbor. In the long run, the RORQ agent learns to identify its optimal forwarding neighbor [2]. The RORQ protocol modifies the standard Q-learning reward function by incorporating factors such as the residual energy of the node and the success rate of packet delivery. However, it does not consider node mobility as part of the reward calculation, which can significantly affect performance in highly dynamic environments. Furthermore, if a selected neighbor node has insufficient energy, it may refuse to forward the packet, resulting in packet loss or increased delay. Another limitation of the RORQ protocol lies in the initial construction of the Candidate Forwarder List (CFL), which includes all neighboring nodes within communication range. This unfiltered inclusion increases the complexity of the agent's learning process and prolongs the convergence time required to identify optimal forwarding nodes. Although the RORQ agent eventually learns to select the optimal neighbor over time, the initial inefficiencies can impact performance, particularly in networks with high node mobility.

The epsilon-greedy strategy used by RORQ protocols and in recent reinforcement learning-based research often fails to maintain a balanced trade-off between exploration and exploitation [2, 13, 14]. A recent study has identified

this limitation as a fundamental challenge in reinforcement learning [15]. As highlighted in the study, enhancing policy learning requires effective cooperation to minimize the risk of wasting resources on unproductive exploratory actions. This challenge becomes even more pronounced in multi-agent environments, where coordination among agents is critical due to the non-stationary nature of the environment and the need for either cooperation or competition [15]. Based on these findings, the proposed method enables each node—modeled as an agent in a MANET—to cooperatively share its mobility speed variations with neighboring nodes. This exchange facilitates more informed and efficient exploration, thereby reducing unnecessary resource consumption. Moreover, agent cooperation significantly improves scalability, and our method has demonstrated strong performance in network scenarios with node densities ranging from 200 to 1000 nodes.

Energy and Signal Strength Q-learning AODV (ESQAODV) is an enhancement over the traditional AODV protocol that incorporates reinforcement learning to optimize route selection in Mobile Ad Hoc Networks (MANETs). The protocol leverages Q-learning to select forwarders that minimize energy consumption and end-to-end delay while improving the packet delivery ratio (PDR) and overall network lifetime.

The decision-making process of the reinforcement learning agent in ESQAODV is primarily governed by two parameters: the residual energy of neighboring nodes and their signal strength. By integrating these parameters into the Q-learning reward function, the protocol constructs energy-efficient forwarding paths.

Although ESQAODV demonstrates a significantly improved packet delivery ratio, it has limitations in highly dynamic environments due to the lack of mobility-awareness in its learning model. The absence of node mobility pattern prediction leads to suboptimal route selection in scenarios with frequent topological changes, resulting in increased end-to-end delays.

The evaluation of ESQAODV is conducted under a low mobility setting, with a maximum node speed of 10 m/s. Despite the stable conditions, the protocol reports significant end-to-end delays, highlighting the need for mobility-adaptive enhancements.

Overall, ESQAODV contributes to energy-efficient routing but lacks dynamic adaptability, which is critical in realistic MANET deployments [13].

Apart from the challenges introduced by node mobility, the cooperation of each node in relaying packets to neighboring nodes until they reach their destination is also crucial for enhancing network lifetime. Networks formed by privately owned nodes tend to exhibit higher levels of cooperation compared to public networks. One of the primary reasons for the non-cooperative behavior of public nodes is their energy concern [16].

In military drone operations based on Unmanned Aerial Vehicles (UAVs) equipped with GPS and IoT devices, location information is sent to a centralized database via the internet. In such scenarios, nodes cooperate with each other to share their location, mobility, and energy information, thereby forming a private and highly cooperative network [17]. These networks require high-computing IoT devices to be installed on the nodes, along with a centralized controller for managing all nodes in the private network.

However, in the proposed method, nodes form a private network in a decentralized scenario. Within this network, mobility and energy information are shared among nodes to enhance the agent's decision-making abilities. Based on mobility speed and energy levels, each node's agent constructs a Candidate Forwarder List (CFL), as described in Algorithms 2 and 3. The CFL ensures that only nodes whose mobility and operational distances fall within the transmission range, and whose energy levels exceed 25%, are selected.

In the Delay-Aware Estimated Transmission Rate (DA-ETR) opportunistic routing protocol, mobile nodes are required to establish a link with mobile sink nodes. Data forwarding to other sink nodes occurs only through these mobile sink nodes. The best forwarding node is selected based on the delay evaluated from intermediate nodes. To enable end-to-end communication between source and destination nodes, each node stores the relaying packet in its buffer until a link is established with a mobile sink node, which ultimately increases the overall transmission delay. Although this method effectively manages node buffer levels, it fails to consider their energy levels. As a result, there is a possibility of including nodes with low energy in the Candidate Forwarder List [14], which could reduce the overall network lifetime.

The Q-learning-based Smart Clustering Routing Protocol (QSCR) used in Flying Adhoc Networks utilizes metrics to identify nodes with similar behavior. Node energy and speed similarity are among the criteria for selecting cluster nodes. The Q-learning process is employed to optimize the metric values. Cluster head selection, which introduces overhead in this protocol, is based on the merit value messages shared by neighboring nodes. In addition to hello packets, nodes with better merit values send a leader message to their neighboring nodes. Once the transmitted leader message is accepted, neighboring nodes send an acceptance message [18]. After the cluster head is determined, another process selects gateway nodes for inter-node communication. This method attempts to establish a centralized network infrastructure before the transmission of data traffic, which creates additional overhead.

The proposed method adopts a single-cluster formation strategy, wherein the current node groups with neighboring nodes located within a 250 m radius. These nodes

are included in the First Forwarder List (FFL), as briefly discussed in Section 3.3. This approach results in minimal computational overhead of $O(1)$. The primary advantage of the proposed single-clustering technique lies in its reduced complexity for node selection, as all nodes reside within a limited range, enabling faster and more efficient decision-making. Moreover, it eliminates the need for frequent re-evaluation of nodes, thereby further reducing computational overhead compared to protocols such as QSCR.

Devices with limited computational power often rely on a centralized entity known as the Mobile Edge Computing (MEC) server to handle their computational tasks. Each MEC server manages a specific cell, and when mobile devices move between cells, they must re-establish an association with the new MEC server. Devices that require computational assistance offload their data to the MEC server to which they are currently associated. A major challenge arises during inter-cell mobility, as the connection with the previous MEC server is lost, and any pending computational tasks must be handed over to the newly associated MEC server. This frequent switching becomes particularly problematic in high-mobility scenarios.

To address this issue, the Mobility-Aware Online Task Offloading (MOTO) framework has been proposed. MOTO employs a Long Short-Term Memory (LSTM)-based algorithm to predict device mobility and a Dueling Double Q-Network (Dueling DDQN)-based algorithm for efficient MEC server grouping [19].

In contrast, the proposed method computes the average mobility speed of a mobile device based on its 10 most recent movements, with a time complexity of $O(n \log n)$. This computation is performed locally on the mobile device, eliminating the need to rely on a centralized MEC server for association management. The computed average mobility speed is then shared with neighboring nodes.

FL-AMM (Federated Learning Augmented Map Matching) uses a machine learning model for map matching [10], which is employed to accurately place a moving entity on the appropriate roadways in the map. The machine learning models are trained with domain-specific data. When applied in environments that were not included in the training data, this can often lead to overfitting. The next challenge is the inclusion of a balanced dataset for machine learning to avoid confusion. Rigorous testing and constant monitoring of model performance are required to address this limitation. However, this approach significantly reduces the control message overhead.

Based on the research gaps identified in the related work, the proposed method addresses these challenges by demonstrating how balanced exploration can be achieved when agents have sufficient mobility information. A detailed explanation of the proposed method is provided in Section 3.

3 Proposed method

Nodes incorporated into the MANET network share their energy levels and average node mobility v_{ai} , which is calculated based on the most recent $n = 10$ movements using Eq. 5. The distance to neighboring nodes is hypothetically estimated using RSSI-based distance estimation [8, 9]. For simulation purposes, the proposed method uses 3D coordinates to represent the positions of nodes. All nodes operate under the IEEE 802.11ax standard at a frequency of 2.4 GHz, with a maximum transmission range of 250 meters.

The buffer space at each node is assumed to be unlimited, based on the assumption that packets are immediately deleted once they are successfully forwarded to a neighboring node.

The operating principles of the proposed algorithm are organized into four key components: **Instantaneous Average Node Mobility Calculation (3.1)**, **3D Mobility Model (3.2)**, **Candidate Forwarder List (CFL) (3.3)**, and **Balanced Q-learning (3.4)**.

3.1 Instantaneous average node mobility calculation (m/s)

Each node in the MANET shares its instantaneous average node mobility with its neighbors. For each second, the node's mobility is recorded in a list and sorted in descending order. To preserve the current mobility status, the total length of the list is truncated to 10 by removing the oldest mobility records.

Mobility positions are recorded every second, along with their corresponding timestamps and positional coordinates. A general representation of each position record is shown in Eq. 1 below:

$$P_i = (t_i, x_i, y_i, z_i) \quad (1)$$

Let $P_i = \{P_1, P_2, \dots, P_n\}$ represents its n positional updates of the current node. The Euclidean distance d_i between two consecutive positions (x_i, y_i, z_i) and $(x_{i-1}, y_{i-1}, z_{i-1})$ is calculated using Eq. 2

$$d_i = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2 + (z_i - z_{i-1})^2} \quad (2)$$

The temporal difference between two consecutive positions P_i & P_{i-1} is calculated using Eq. 3.

$$\Delta t_i = t_i - t_{i-1} \quad (3)$$

Instantaneous speed v_i (m/s) is calculated using Eqs. 2 and 3.

$$v_i = \frac{d_i}{\Delta t_i} \quad (4)$$

From Eq. 4, the average speed of the current node, denoted as v_{ai} , is calculated over n position updates, where $n = 10$:

$$v_{ai} = \frac{1}{n-1} \sum_{i=2}^n v_i \quad (5)$$

Algorithm 1 Calculate Instantaneous Speed of the Current Node n_i from Recent Positional Coordinates.

Require: List of position records with 3D coordinates (x, y, z) and timestamp *Time*

Ensure: Truncated position list and instantaneous speed v_{ai} of the current node

```

1:  $P_i \leftarrow$  Convert position array to list
2: if  $P_i$  is not empty then
3:    $P_i \leftarrow$  Sort by Time in descending order
4:   if Length of  $P_i > 10$  then
5:      $P_i \leftarrow$  Truncate to the latest 10 positions
6:   end if
7: end if
8: Convert  $P_i$  back to NumPy array
9:  $df \leftarrow$  Convert position array to DataFrame
10:  $df[Time] \leftarrow$  Convert to datetime format
11:  $df[dx] \leftarrow$  Compute difference in  $x$ 
12:  $df[dy] \leftarrow$  Compute difference in  $y$ 
13:  $df[dz] \leftarrow$  Compute difference in  $z$ 
14:  $df[distance] \leftarrow \sqrt{dx^2 + dy^2 + dz^2}$ 
15:  $\Delta t_i \leftarrow$  Compute time difference in seconds
16:  $v_i \leftarrow distance / time\_diff$ 
17: Drop rows with missing values
18:  $v_{ai} \leftarrow$  Compute mean of  $v_i$ 
19: return  $v_{ai}$  (rounded to the nearest integer) as instantaneous speed

```

The total time complexity of the algorithm (1) is $O(n \log n)$ and the space complexity is $O(n)$. Sharing its instantaneous node mobility helps other node agents in the Q-learning process. The proposed method uses these values in the Q-learning reward calculation as described in Eqs. 18 and 20.

3.2 3D mobility model

The mobility model chosen for the simulation is the 3D Steady-State Random Waypoint Mobility Model. Based on the analysis of various mobility models suitable for MANETs, it is evident that the Steady-State Random Waypoint model provides optimal performance for this type of scenario [20]. The movement of nodes is adjusted using a

uniform random movement generator, based on the current positional coordinates (x_c, y_c, z_c) of each node.

The maximum simulation area is confined within a cube of dimension $\text{space_dim} = (1000 \times 1000 \times 1000)$ meters. The target position coordinates of the nodes are constrained within the limits defined in Eq. 6. The placement of nodes in the 3D space for $n = 100$ nodes is illustrated in Fig. 1.

$$0 \leq x_t \leq \text{space_dim}, \quad 0 \leq y_t \leq \text{space_dim}, \quad 0 \leq z_t \leq \text{space_dim} \quad (6)$$

Here, (x_t, y_t, z_t) represent the target position coordinates of the node. The distance d_m between the current and target positions is computed using Eq. 7:

$$d_m = \sqrt{(x_t - x_c)^2 + (y_t - y_c)^2 + (z_t - z_c)^2} \quad (7)$$

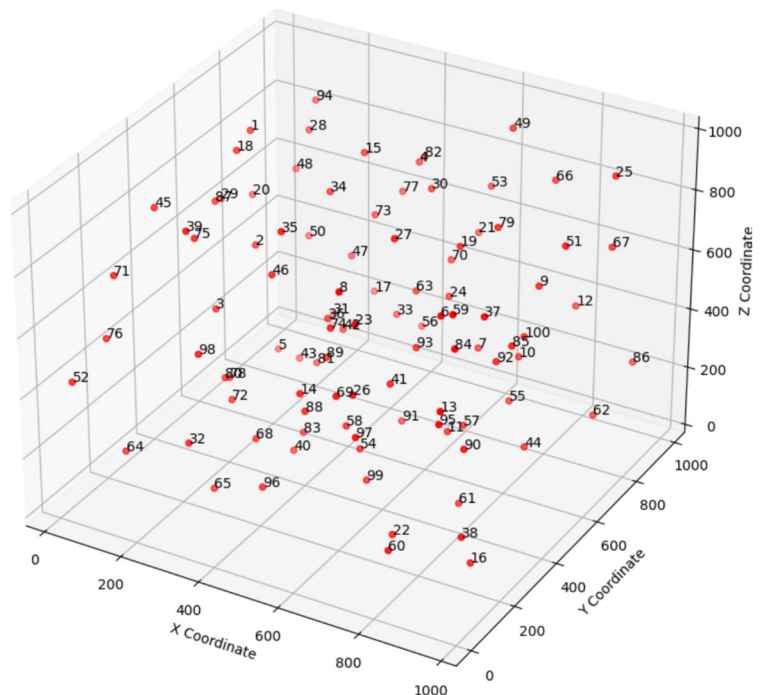
The directional distance vectors d_x , d_y , and d_z are then calculated using Eq. 8:

$$d_x = \frac{(x_t - x_c)}{d_m}, \quad d_y = \frac{(y_t - y_c)}{d_m}, \quad d_z = \frac{(z_t - z_c)}{d_m} \quad (8)$$

The movement of the nodes is governed by a constant velocity v_m (in m/s). Since the movement is in a steady state, the new position of the node is updated based on its current position, as shown in Eq. 9:

$$x_c = x_c + v_m \cdot d_x, \quad y_c = y_c + v_m \cdot d_y, \quad z_c = z_c + v_m \cdot d_z \quad (9)$$

Fig. 1 Position of Node in 3D
Space $n=100$



3.3 Candidate forwarder list (CFL)

The initial step in traditional opportunistic routing involves selecting neighboring nodes, collectively referred to as the *Candidate Forwarder List (CFL)*. As per the IEEE 802.11ax standard, a maximum detection range of 250 meters is used to identify neighboring nodes. All nodes within this range are included in the conventional CFL. In reinforcement learning-based approaches, this CFL is provided to an agent node, which then determines the most suitable forwarding node.

The agent selects a node from the CFL and evaluates its Q-value—a process known as *exploration*. However, a significant challenge arises when the selected node is suboptimal, such as having low residual energy, high mobility, or a tendency to move out of communication range. In such cases, the agent's learned Q-value may become inaccurate or outdated over time.

For instance, as illustrated in Fig. 2, if node 1 selects node 2 or node 5, the decision may lead to instability. Node 2, due to its high mobility, might quickly move out of range, resulting in a broken link. Meanwhile, node 5, with low residual energy, may soon exhaust its battery and leave the network. In both scenarios, the Q-values associated with these nodes become obsolete, forcing node 1 to reinitiate the exploration phase to identify a new optimal forwarder. This repeated exploration process increases convergence time and overall end-to-end delay.

To address this challenge, it is crucial that the CFL introduced to the agent is optimal. This involves excluding nodes with low energy or high mobility, thereby reducing unnecessary exploration and improving convergence efficiency.

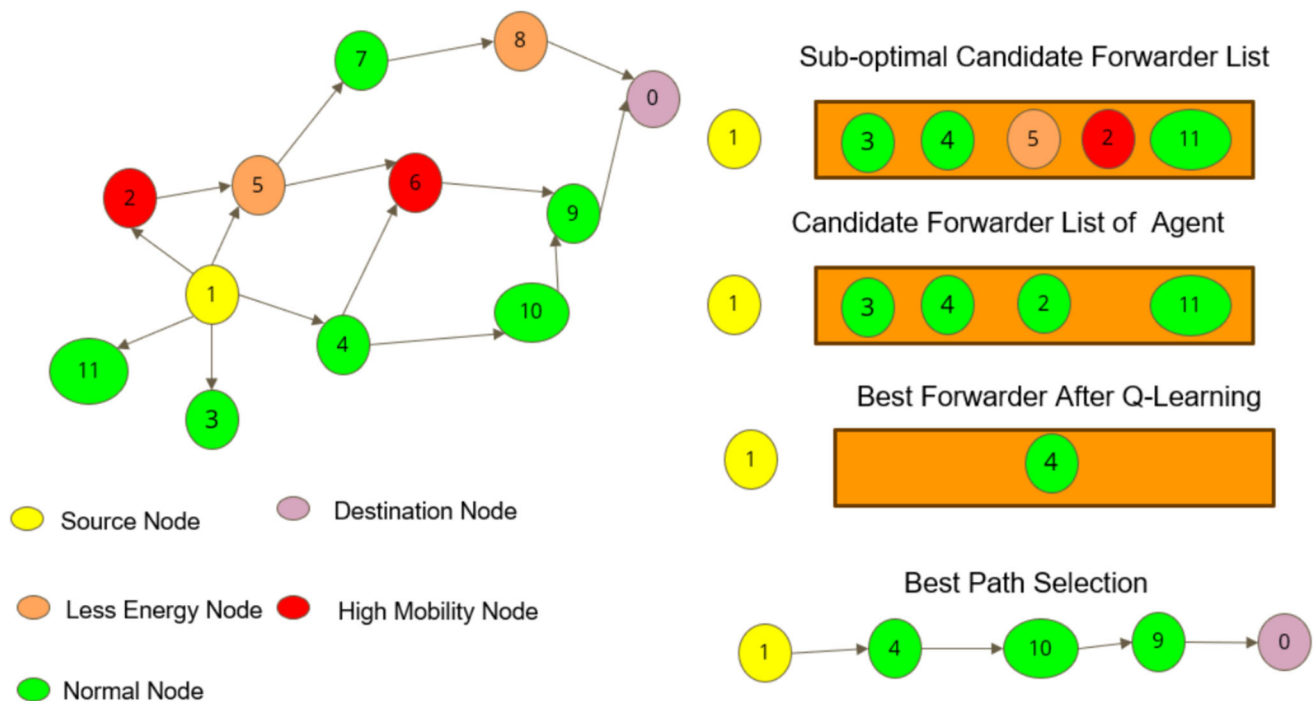


Fig. 2 New Candidate Forwarder Selection

The Candidate Forwarder List (CFL) selection is a two-step process. In the first step, nodes located within a proximity of 250 meters and residual energy greater than 25% are included in the First Forwarder List (FFL). This step is continuously executed, allowing each node to periodically scan its environment and update the FFL to ensure optimal participation in the network.

The second step is triggered when a node intends to transmit a packet to the destination. At this point, the agent selects a node from the FFL and computes a *threshold distance*, defined as the sum of the selected node's distance and its current speed as illustrated in Fig. 3. This threshold distance determines whether the node should be considered for exploration.

As illustrated in Fig. 4, if the threshold distance exceeds 250 meters, the *choice_selector* flag is set to false, prompting the agent to discard the current node and select another forwarder. Conversely, if the threshold distance is below 250 meters, the node is considered suitable for exploration. Thus, the agent's decision to explore is directly influenced by the speed of the neighboring nodes, ensuring that highly mobile or distant nodes are excluded to maintain network stability.

To construct the Intermediate First Forwarder List (IFFL) and the First Forwarder List (FFL), as illustrated in Fig. 4, a node must interact with its environment to identify neighboring nodes. In real-world implementations, when electromagnetic signals are received from nearby nodes, the

receiving node measures the Received Signal Strength Indicator (RSSI). Based on the RSSI values, it estimates the distance to each neighboring node and infers their energy levels [6–9].

To prepare an energy-efficient Intermediate First Forwarder List (IFFL), nodes with residual energy less than 25% are filtered out. The resulting IFFL is then processed using the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm to generate the First Forwarder List (FFL) as illustrated in Fig. 3.

In the proposed simulation environment, positional coordinates such as (x, y, z) are utilized to build the First Forwarder List (FFL). If a node n_i intends to identify its neighbors, it calculates the Euclidean distance to each neighboring node n_j using the DBSCAN algorithm. This Euclidean distance can be correlated with the estimated distance obtained from RSSI measurements in real-world scenarios.

The Euclidean distance between the current node n_i and neighbor node n_j is calculated as:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \quad (10)$$

where, (x_i, y_i, z_i) are the positional coordinates of current node n_i , and (x_j, y_j, z_j) are the positional coordinates of neighbor node n_j

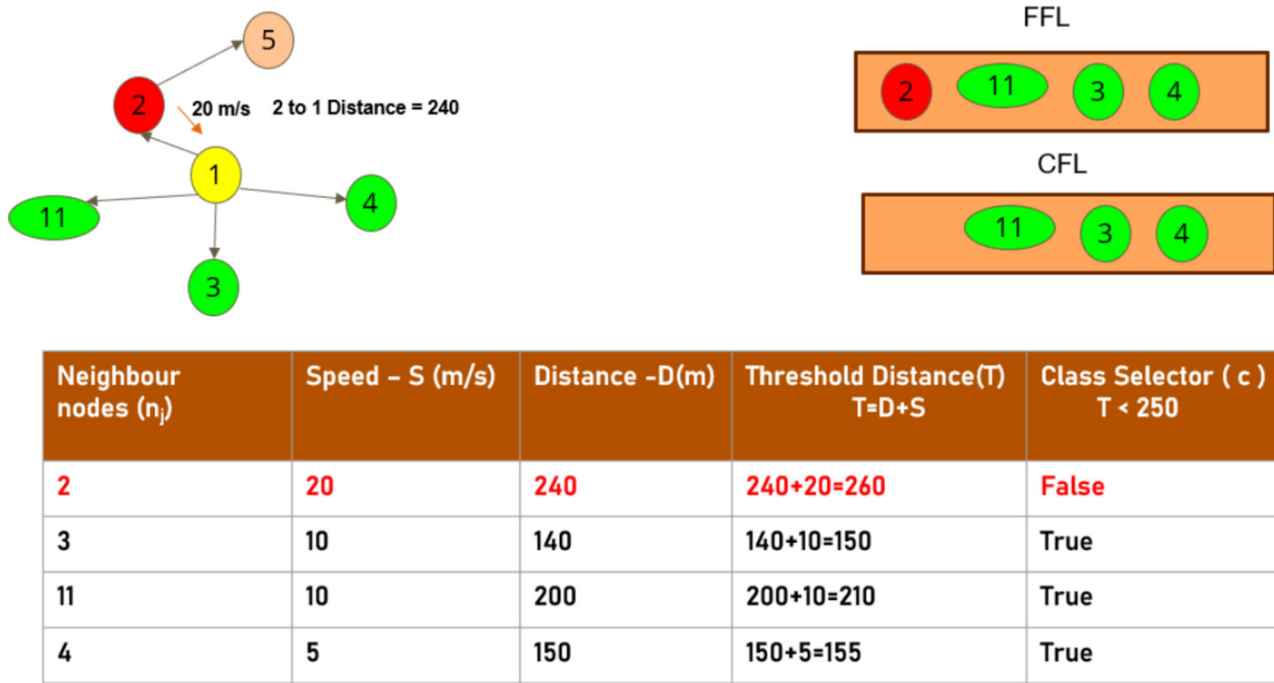


Fig. 3 CFL Selection

DBSCAN is applied to the Intermediate First Forwarder List (IFFL) to identify cluster structures as shown in Fig. 4. Neighboring Nodes n_j are grouped into single cluster based on their spatial proximity, where core points are assigned with cluster index as 0, and outlier or noise points are

labeled as -1 . The implementation follows the standard `scikit-learn` library [21].

$$\text{labels} = \text{DBSCAN}(\epsilon = l, \text{min_samples} = m, \text{metric} = \text{'euclidean'}).fit(\text{IFFL}) \quad (11)$$

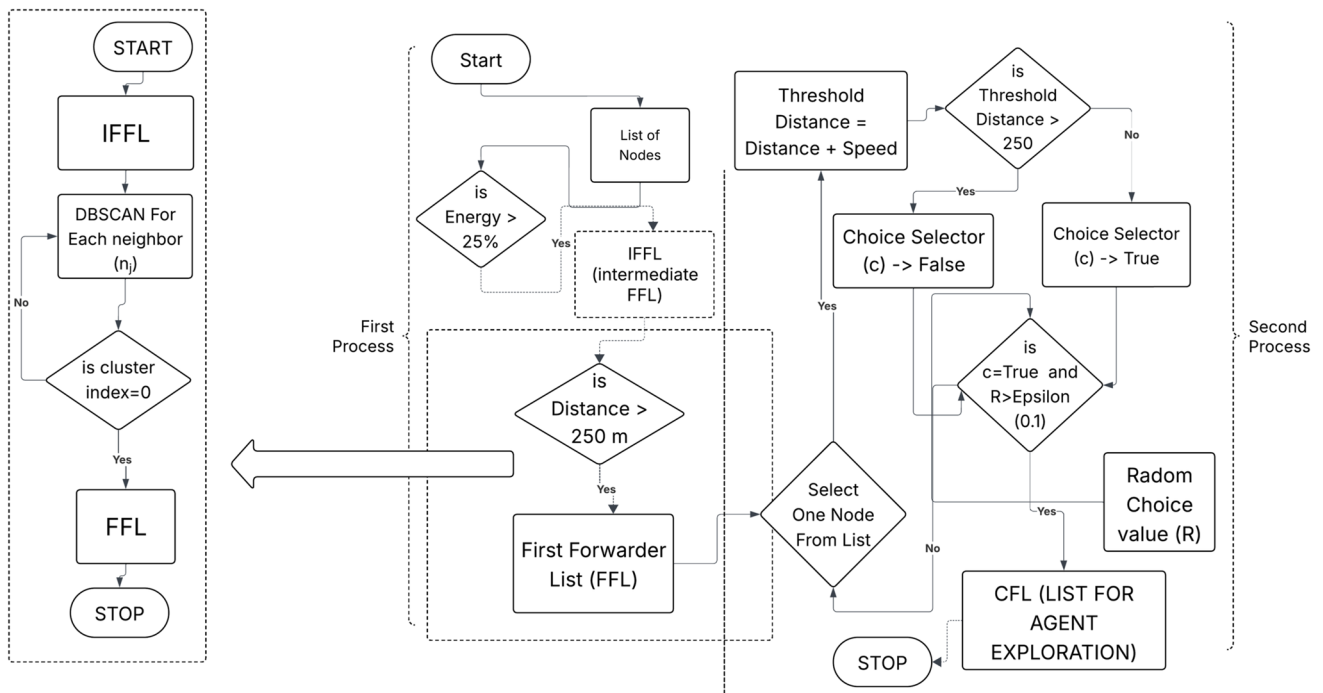


Fig. 4 Best Forwarder Selection Process

where $l = 250\text{ m}$ is the maximum distance for nodes n_i and n_j to be considered part of the same cluster, $\text{min_samples} = 2$ is the minimum number of points required to form a dense region, and $\text{metric} = \text{euclidean}$ is the distance metric.

If both nodes belong to the same cluster (i.e., $\text{labels}[1] = 0$), node n_j is considered a potential forwarder for node n_i and append to the FFL as mentioned in Eq. 12.

$$\mathcal{F}(n_i) = \{n_{ij} \mid E(n_{ij}) > 0.25 \text{ and } d(n_i, n_{ij}) < 250\text{ m}, j = 1, 2, \dots, k\} \quad (12)$$

$\mathcal{F}(n_i)$: Set of First Forwarder List (FFL) of node n_i .

n_{ij} : The j th neighbor of node n_i that qualifies as a forwarder.

$E(n_{ij}) > 0.25$: Ensures that the neighbor has more than 25% of its residual energy.

$d(n_i, n_{ij}) < 250\text{ m}$: Ensures that the neighbor is within a 250 meter distance.

$\text{Index } j$: Denotes that there are potentially multiple forwarders (i.e., $j = 1, 2, \dots, k$) for each node n_i .

It is important to analyze the computational overhead involved in the selection of the FFL. The simulation is conducted on a set of nodes n_i , where $i = 1, 2, \dots, N$. From the perspective of a single node, let us consider node n_1 . The computational overhead of the DBSCAN clustering algorithm is specifically analyzed for this node.

For distance calculations, node n_1 compares itself with all other nodes n_j , where $j = 1, 2, \dots, K$, requiring $O(N)$ time. DBSCAN clustering is applied pairwise, such as n_1 with n_{11} , n_1 with n_{12} , up to n_{1K} . Each such pairwise clustering operation takes $O(1)$ time.

When generalized to K neighbors, the DBSCAN clustering computation becomes $O(K \log K)$, where $K \leq N$ is the number of neighboring nodes within the defined distance threshold.

The cost of updating the First Forwarder List (FFL) per valid forwarder is constant, i.e., $O(1)$. Thus, the overall computational complexity for preparing the First Forwarder Selection from a single node perspective is:

$$O(N \log N) \quad (13)$$

The Candidate Forwarder List (CFL) is generated based on the mobility information shared by neighboring nodes, as described in Eq. 5. For each neighbor, a threshold distance T_{ij} is computed in Eq. 14 as the sum of the estimated distance d_{ij} , calculated using Eq. 10, and the neighbor node's mobility speed v_{ij} , as discussed in Eq. 17.

$$T_{ij} = d_{ij} + v_{ij} \text{ for all } j \neq i, j = 1, 2, \dots, K \quad (14)$$

If the threshold distance T_{ij} exceeds 250m, the choice selector (c) flag is set to false as illustrated in Fig. 4,

and the corresponding node is removed from the First Forwarder List (FFL). Conversely, if the threshold distance is less than or equal to 250m, the node is retained as a reliable forwarder and included in the Candidate Forwarder List (CFL) for agent-based exploration.

For example, consider node 2 as a neighbor of node 1, as illustrated in Fig. 3, which is a continuation of the network topology shown in Fig. 2. Let the mobility speed of node 2 be 20 m/s, and its estimated distance to node 1 be 240 m. Although the Euclidean distance is within the communication range, the threshold distance becomes:

$$T = 240 + 20 = 260\text{ m}$$

Since this exceeds the 250 m threshold, node 2 is excluded from the FFL. Only those nodes with threshold distances less than or equal to 250m are retained in the CFL for further processing.

It is also important to note that in the simulation setup, the nodes' energy levels decrease gradually by a factor of 0.05.

Since the node's positions are constantly changing, it is necessary to update the FFL frequently. Whenever a nodes FFL is updated, it is assumed that the node energy is reduced by a factor of 0.05. The updated node energy is shown in Eq. 15.

$$E_{\text{new}} = E_{\text{old}} - \Delta E m \quad (15)$$

E_{old} is the current energy level of the node, $\Delta E = 0.05$ is the energy consumed during the refresh, E_{new} is the updated energy level.

Algorithm 2 DBSCAN FFL.

Require: $n_i, n_j \in \mathcal{N}$, $\epsilon = 250$, $\text{MinPts} = 2$, $\text{metric} = \text{euclidean}$

Ensure: Forwarders for each node based on DBSCAN clustering

```

1: now ← current datetime
2: for each  $n_i$  in  $\mathcal{N}$  do
3:   for each  $n_j$  in  $\mathcal{N}$ , where  $i \neq j$  do
4:     if  $n_j.\text{energy} > 25$  then
5:        $\text{IFFL} \leftarrow \text{reshape}([n_i.x, n_i.y, n_i.z, n_j.x, n_j.y, n_j.z],$ 
6:          $(-1, 3))$ 
7:        $\text{db} \leftarrow \text{DBSCAN}(\epsilon, \text{MinPts}, \text{metric}).\text{fit}(\text{scan\_list})$ 
8:        $\text{labels} \leftarrow \text{db.labels\_}$ 
9:       if  $\text{labels}[1] = 0$  then
10:         $\text{FFL} \leftarrow n_i.\text{db\_scan\_forwarder}.\text{append}(j, n_j,$ 
11:           $x_j, y_j, z_j, t_j)$ 
12:        end if
13:      end for

```

3.4 Balanced Q-learning

When a node intends to send a packet, it employs a Modified Reinforcement Learning approach, specifically the

Q-learning algorithm. Prior to interacting with the environment, the agent selects a node from the First Forwarder List (FFL), as defined in Eq. 12, to determine whether the selected node is suitable for exploration. This decision-making process is elaborated in Section 3.3. The learning behavior of the agent and its interaction with the environment are illustrated in Fig. 5.

FFL (Forwarder Filter List) ensures that the list contains only highly energized neighboring nodes n_j , with energy levels greater than 25%, as shown in Eq. 12. In the current approach, the agent's decision-making is optimized based on the mobility information shared by the neighboring nodes n_j , which is computed using Eq. 5. To balance exploration and exploitation, the Epsilon-Greedy strategy is employed. A customized version of the Epsilon-Greedy approach is proposed in this method. The modified form of Eq. 5, adapted to the current scenario, is presented in Eqs. 16 and 17.

Each neighboring node $n_j \in N_i$, where $j \neq i$, computes its average speed $\bar{v}_j$ over its recent 10 positions as:

$$\bar{v}_j = \frac{1}{9} \sum_{t=2}^{10} v_j^{(t)} \quad (16)$$

Then, node n_i receives the average speed from each neighbor n_j , as shown in Eq. 17, and incorporates it into the reward calculation described in Eq. 20.

$$v_{ij} = \bar{v}_j, \quad \text{for all } j \neq i, j = 1, 2, \dots, K \quad (17)$$

$\Rightarrow$ Neighbor mobility information of n_i : $\{v_{ij} \mid j \neq i, j = 1, 2, \dots, K\}$

In a limited exploration-time setup with $\varepsilon_i = 0.15$, it is crucial to provide only valid neighbor nodes to the agent for

exploration. In the proposed method, if the selected neighbor node's threshold distance exceeds 250 m, the agent is forced to select the next valid neighbor node from the First Forwarder List (FFL), as shown in Fig. 5. This mechanism ensures that the node utilizes its exploration phase efficiently by interacting only with valid candidates. The detailed process for selecting a valid neighbor node is explained in Section 3.3.

The agent's exploration begins when the random choice number rc is less than ε_i , following the standard epsilon-greedy strategy. In the proposed method, a choice selector (c) is introduced as an additional decision parameter to influence agent exploration. Exploration is triggered when both the random choice number rc and the choice selector (c) condition are satisfied, as described in Section 3.3 and Algorithm 3.

Let rc be a random number chosen uniformly from the interval $[0, 1]$. The exploration action of the node agent n_{ai} is denoted as a_{ex} . In general, the node's action is determined by the decision rule given in Eq. 18.

$$n_{ai} = \begin{cases} a_{ex} & \text{if } r_c < \varepsilon_i \text{ and } c = (T_{ij} < 250), \\ \text{calculate } T_{ij} \text{ for } j \neq j' & \text{otherwise.} \end{cases} \quad (18)$$

n_{ai} : Represents the action of agent i .

a_{ex} : Action or decision to be executed (for example, a specific action in the system).

r_c : Random Choice Value that is selected in random between 0 and 1.

$\varepsilon_i = 0.15$: Exploration rate of current node i

T_{ij} : Threshold Distance Calculated based on Eq. 14

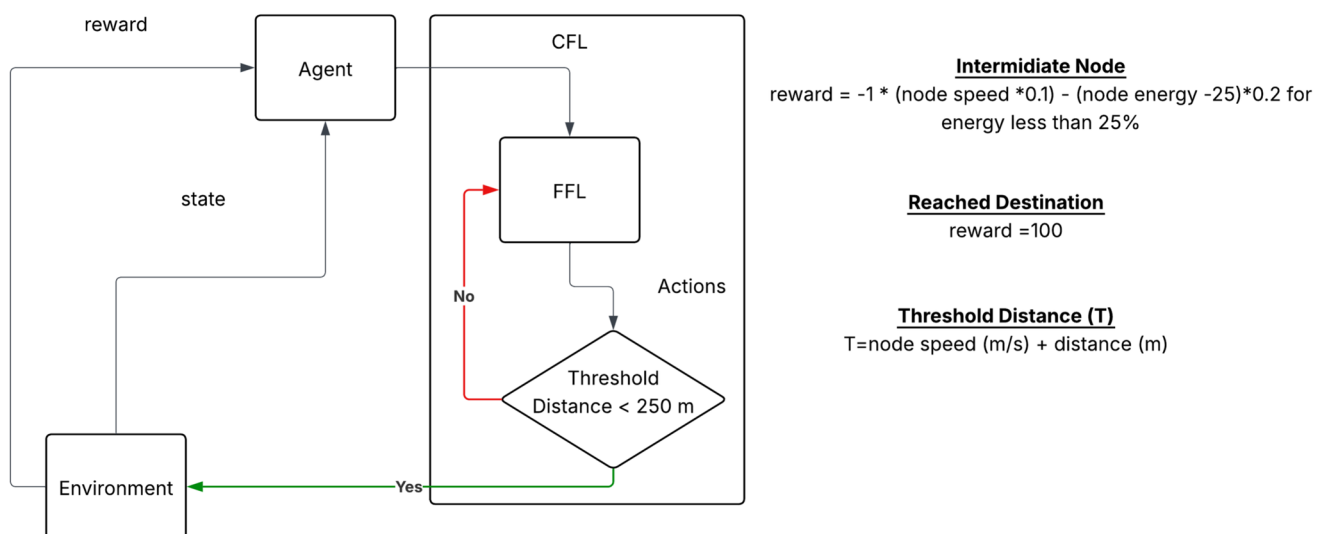


Fig. 5 Mobility Aware Reinforcement Learning Agent

$j \neq j'$: Ensures the calculation applies only to different agents.

The exploration rate is set to $\varepsilon_i = 0.15$ based on the analysis discussed in Section 4.1.

Each node n_i maintains the Q-table based upon Modified Bellman Eq. 19.

$$Q_i^{(j)}(s, a) \leftarrow (1 - \alpha) \cdot Q_i^{(j)}(s, a) + \alpha \left[r_i^{(j)} + \gamma \cdot \max_{a'} Q_i^{(j)}(s', a') \right] \quad \text{for } j = 1, 2, \dots, K; \ j \neq i \quad (19)$$

Algorithm 3 Balanced Q-Learning.

Require: Learning rate $\alpha = 0.1$, Discount factor $\gamma = 0.9$,
Exploration rate $\epsilon = 0.15$

```

1:  $n_i \leftarrow$  current node
2: while True do
3:   if not  $\mathcal{F}(n_i)$  then
4:     return False
5:   end if
6:    $n_j \leftarrow \text{random.choice}(\mathcal{F}(n_i))$ 
7:    $T_{ij} \leftarrow n_{dij} + n_{vij}$  (Threshold)
8:   print:  $T_{ij} < 250$ 
9:   if  $T_{ij} > 250$  or  $n_j$  is None then
10:     $c \leftarrow$  False
11:   else
12:     $c \leftarrow$  True
13:   end if
14:   if  $\text{random.uniform}(0,1) < \epsilon$  and  $c$  then
15:     $n_j \leftarrow \text{random.choice}(\mathcal{F}(n_i))$ 
16:   else if  $\text{random.uniform}(0,1) > \epsilon$  and not  $c$  then
17:    Remove  $n_j$  from  $\mathcal{F}(n_i)$ 
18:     $n_j \leftarrow \text{random.choice}(\mathcal{F}(n_i))$ 
19:   else
20:     $n_j \leftarrow \max(\mathcal{F}(n_i), \text{key} = \lambda f: Q(n_i, f, d))$ 
21:   end if
22:    $r \leftarrow -1 - n_j.\text{speed}() \times 0.1$ 
23:   if  $n_j.\text{energy} < 25$  then
24:     $r \leftarrow r - (n_j.\text{energy} - 25) \times 0.2$ 
25:   end if
26:   if  $n_j.ip == d$  then
27:     $r \leftarrow 100$ 
28:    Append packet to  $n_j$ 
29:     $Q(n_i, n_j, d) \leftarrow r$ 
30:    return True
31:   end if
32:    $Q_{curr} \leftarrow Q(n_i, n_j, d)$ 
33:    $Q_{next} \leftarrow \max(Q(n_j, *, d))$ 
34:    $Q(n_i, n_j, d) \leftarrow (1 - \alpha)Q_{curr} + \alpha(r + \gamma Q_{next})$ 
35:   Decrease packet TTL
36:    $n_i \leftarrow n_j$ 
37: end while

```

Unlike the traditional approach of assigning fixed values for positive and negative rewards, the proposed method computes rewards dynamically by considering the mobility and energy parameters of neighboring nodes. Let the energy of the forwarders, represented in Eq. 12, be

$$e_{ij} = \{e_{i1}, e_{i2}, \dots, e_{ik}\}, \quad \text{where } i \neq j, \ j = 1, 2, \dots, K,$$

and let v_{ij} be calculated based on Eq. 17. Then, the final reward is computed using Eq. 20.

$$r_i^{(j)} = -1 - (v_{ij} \times 0.1) + \begin{cases} (e_{ij} - 25) \times 0.2, & \text{if } e_{ij} < 25 \\ 0, & \text{otherwise} \end{cases} \quad \text{for } j = 1, 2, \dots, K; \ j \neq i \quad (20)$$

4 Simulation setup

Simulation environment is developed using Python libraries and modules. Nodes are placed in a 3D space of $(1000 \times 1000 \times 1000)$ meters with the mobility setup as discussed in Section 3.2. Sklearn [21], Matplotlib [22] and Pandas [23] are used for various operations in Simulations. Pandas is used for mobility data analysis [23] and DBSCAN clustering is used for forwarder selection [21].

Nodes are initialized using configuration objects, where each node is assigned a unique IPv4 address along with 3D positional coordinates (x, y, z) . The movement of nodes is governed by a steady-state Random Waypoint Mobility Model, as detailed in Section 3.2. This model dynamically adjusts node mobility based on a velocity parameter v_m (m/s), as described in Eq. 9.

Energy consumption for each node is computed using Eq. 15, which accounts for energy depletion both during packet transmission and during updates to the Candidate Forwarder List (CFL). Initially, nodes are placed at predefined coordinates (x, y, z) . As the simulation progresses, their positions are continuously updated in accordance with the set mobility speed v_m , ensuring movement within a bounded 3D space of dimensions $(1000 \times 1000 \times 1000)$ meters.

This integration of dynamic mobility and energy modeling allows for realistic simulation of node behavior under varying network conditions, forming a critical foundation for evaluating the proposed routing protocol's performance. To facilitate a comprehensive analysis and comparison of the proposed routing protocol, a dedicated simulation table is presented, summarizing the key parameters and configurations employed during the experiments.

4.1 Optimum exploration rate analysis

The proposed method controls the agent's exploration decision based on a threshold distance calculated from neighboring nodes. Analyzing the relationship between mobility, the distance between source and destination nodes, delay, and the exploration rate helps in understanding the performance behavior of the proposed model. The simulation parameters used for the analysis are presented in Table 2.

Table 2 Simulation parameters for exploration rate analysis in MANET routing

Setup	Parameter
Number of nodes	100
Node Speed	Variable (20—50 m/s)
Nodes with Energy < 25%	10
Number of Packets Transmitted	2000 /s
Mobility Model	Steady State Random Way Point 3D
Wifi Standard	802.11ax
Area	1000 x 1000 x 1000 m
Communication range	250 m
Exploration rate ε_i	Variable (0.1—0.25)

Table 3 Simulation Parameters for Evaluating the Balanced Epsilon-Greedy Strategy

Setup	Parameter
Number of nodes	100
Node Speed	50 m/s
Nodes with Energy < 25%	10
Number of Packets Transmitted	2000 /s
Mobility Model	Steady State Random Way Point 3D
Wifi Standard	802.11ax
Area	1000 x 1000 x 1000 m
Communication range	250 m
Exploration rate ε_i	0.15

In performance analysis, factors such as distance, mobility speed, and node energy levels play a crucial role. Delay typically increases as the distance between the source and destination nodes grows. Similarly, higher mobility speeds can lead to frequent link breakages due to nodes moving beyond the reception detection zone, resulting in increased delay.

In the proposed implementation, the exploration parameter is controlled by the agent's exploration rate ε_i , a random choice rc , and a choice selector (c) as discussed in Eq. 18. The ε -greedy approach is a probabilistic method, where $\varepsilon_i = 0.1$ implies that the agent explores 10% of the time. The proposed method prevents exploration when the threshold distance exceeds 250 m, as described in Section 3.3. Hence, determining the optimum exploration parameter is crucial for balancing exploration and exploitation.

Based on the insights drawn from Fig. 6, the **Distance vs. Delay** graph indicates that delay tends to increase with greater source-destination distance due to the involvement of longer paths. The exploration rate $\varepsilon_i = 0.15$ demonstrates improved delay performance across all distance bins, especially in the mid-range distances, by enabling the agent to choose more adaptive forwarding decisions.

In the **Mobility Speed vs. Delay** graph, delay increases with mobility for most exploration rates, highlighting the impact of frequent route changes. However, at $\varepsilon_i = 0.15$, the delay remains consistently lower across different mobility speeds, indicating that the agent is better able to adapt to topology changes and maintain stable routing performance.

Although the number of nodes with low energy remains constant in the dataset, analysis of the **delay distribution via boxplots** across exploration rates shows that $\varepsilon_i = 0.15$ results in the most consistent and lower delays. This suggests more stable and efficient forwarding behavior, which indirectly reflects better energy utilization and less retransmission.

Therefore, incorporating a threshold-based exploration decision strategy enables the discovery of an optimal exploration rate at $\varepsilon_i = 0.15$, balancing delay, mobility adaptability, and energy efficiency in dynamic MANET environments.

4.2 Balancing exploration and exploitation: An analysis of the proposed epsilon-greedy method

To evaluate the performance of the customized epsilon-greedy strategy, simulations were conducted using the

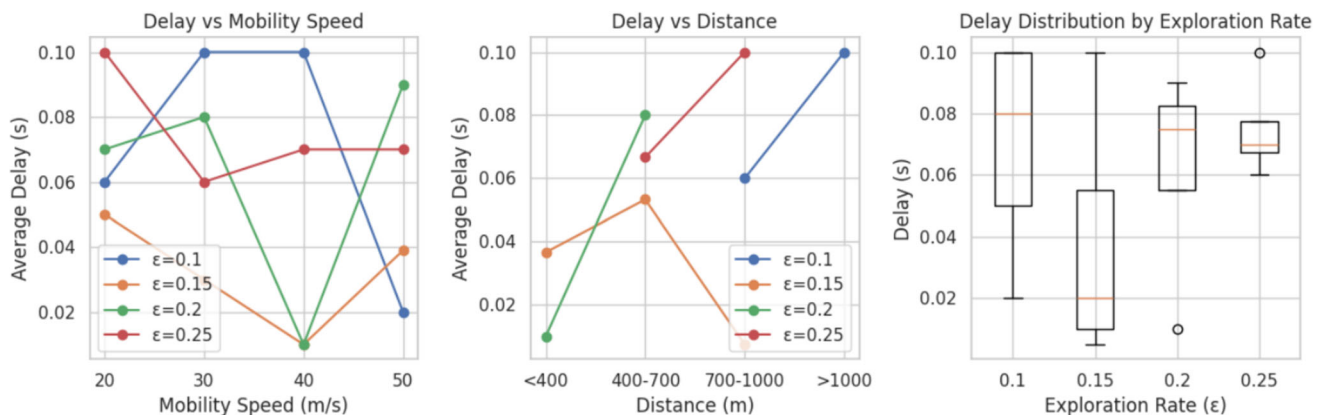
**Fig. 6** Impact of Exploration Rate on Routing Delay with Respect to Distance and Mobility Speed

Table 4 Reinforcement learning agent: exploration, exploitation, and total reward

RL Metric	Value
Number of Exploration Actions	13,048
Number of Exploitation Actions	12,639
Total Reward Accumulated	185,628.90

parameters listed in Table 3. Based on the analysis presented in Section 4.1, the optimum exploration rate $\varepsilon_i = 0.15$ was selected.

The performance metrics observed from our epsilon-greedy strategy, including the number of exploration and exploitation actions as well as the total accumulated reward, are summarized in Table 4.

The total number of actions performed by the agent is determined using the following expression (Eq. 21):

$$\begin{aligned} \text{Total Actions} &= \text{Exploration} + \text{Exploitation} = 13,048 \\ &+ 12,639 = 25,687 \end{aligned} \quad (21)$$

The ratio of exploration to exploitation is calculated as shown in Eq. 22:

$$\text{Ratio} = \frac{\text{Exploration}}{\text{Exploitation}} = \frac{13,048}{12,639} \approx 1.033 \quad (22)$$

This ratio indicates a balanced approach, suggesting that the strategy effectively balances exploration and exploitation. The expected number of exploitations and explorations based on the epsilon-greedy strategy can be calculated using Eqs. 23 and 24, respectively.

$$\begin{aligned} \text{Expected Exploitations} &= (1 - \varepsilon_i) \times \text{Total Actions} \\ &= 0.85 \times 25,687 \approx 21,834 \end{aligned} \quad (23)$$

$$\begin{aligned} \text{Expected Explorations} &= \varepsilon_i \times \text{Total Actions} \\ &= 0.15 \times 25,687 \approx 3,863 \end{aligned} \quad (24)$$

The comparison with the actual values reveals that the strategy placed a stronger emphasis on exploration (13,048) than on exploitation (12,639), indicating the adaptability of the epsilon-greedy strategy in exploring new actions beyond initial expectations.

These analytical results are compared using the graph shown in Fig. 7, which illustrates how the proposed epsilon-greedy strategy effectively balances exploration and exploitation at the optimal exploration rate of $\varepsilon_i = 0.15$.

4.3 Network life time tradeoff

The end-to-end delay increases slightly as the number of nodes excluded from the FFL using DBSCAN, rising from 0.04 seconds to 0.06 seconds. This is because excluding nodes reduces the number of available routing paths, which forces the agent to select longer or less optimal routes. Although this increase is noticeable, overall network connectivity is still maintained.

The packet delivery ratio (PDR) decreases from 0.98 to 0.85 as more low-energy nodes are excluded. This reduction is due to decreased redundancy in the available optimal paths. The lower PDR indicates a slight decline in the reliability of data transmission when fewer low-energy nodes are available in the forwarding list.

An important observation from this analysis is that the network lifetime increases as more low-energy nodes are excluded. Removing these nodes reduces the overall energy consumption, allowing the network to remain operational for a longer duration. This tradeoff is clearly illustrated in Fig. 8.

The simulation setup for the network lifetime tradeoff analysis is shown in Table 5

Fig. 7 Comparison of Actual vs. Expected Exploration and Exploitation Actions at $\varepsilon_i = 0.15$

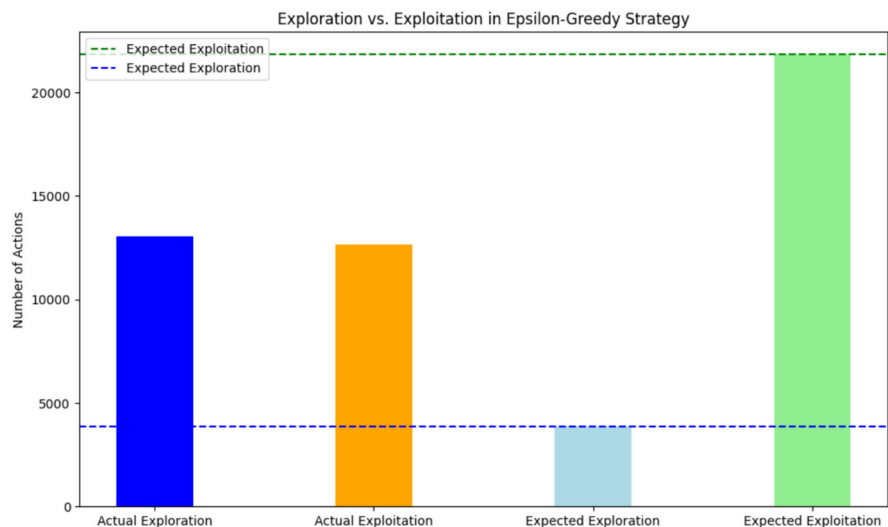
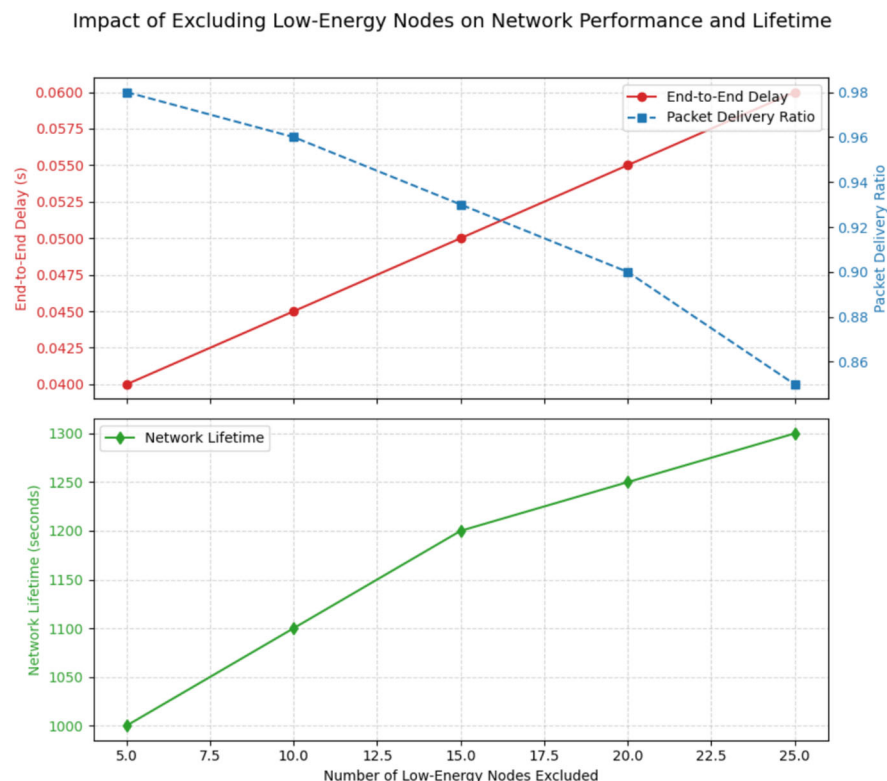


Fig. 8 Impact of Excluding Low-Energy Nodes on Delay, Packet Delivery Ratio, and Network Lifetime



4.4 Scalability analysis and delay performance with fixed exploration rate

The simulation setup for the scalability analysis is shown in Table 6

With the exploration rate fixed at 0.15, the delay incurred by the proposed method under varying node densities is analyzed. Based on the insights drawn from the Fig. 9, delay fluctuations are observed at node densities of 200 and 500 nodes, where the delay drops to 0.07 seconds and 0.03 seconds, respectively. For other node densities, the delay remains stable at approximately 0.1 seconds. This analysis clearly reveals that the delay does not increase with the rise in node

density, indicating that the proposed method is highly scalable without significant degradation in delay performance. The low delay observed at a node density of 500 further highlights the routing efficiency of the proposed algorithm.

The analysis also suggests that the simulation was conducted in a highly dynamic environment. In the distance vs. node analysis, the distance between the source and destination is measured as 956 m at 200 nodes and 256 m at 500 nodes. The shortest delay recorded in the 500-node scenario can be attributed to the shorter source-destination distance.

The system demonstrates scalability by maintaining stable delay even in the 1000-node density scenario. The relationship between delay and distance reflects the dynamic nature

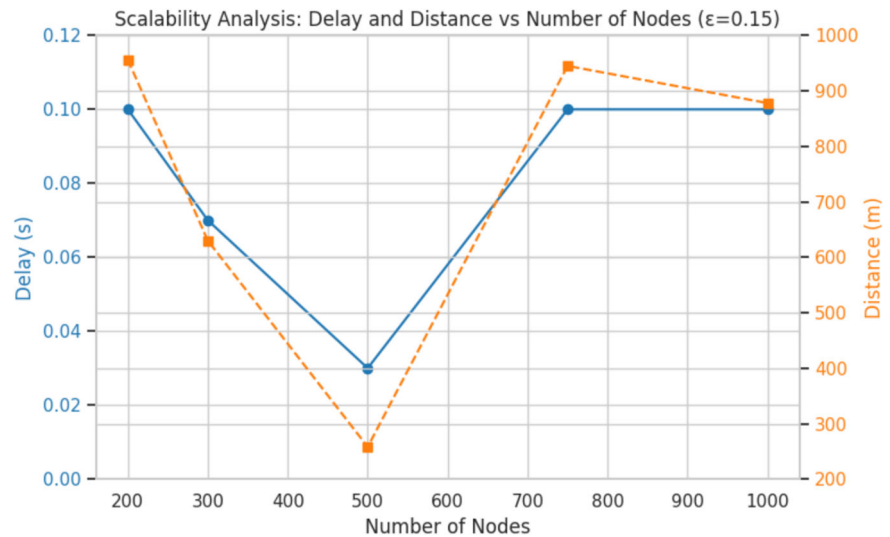
Table 5 Simulation Parameters for Network Lifetime Tradeoff Analysis

Setup	Parameter
Number of nodes	100
Node Speed	50 m/s)
Nodes with Energy < 25%	Variable(5—25)
Number of Packets Transmitted	2000 /s
Mobility Model	Steady State Random Way Point 3D
Wifi Standard	802.11ax
Area	1000 x 1000 x 1000 m
Communication range	250 m
Exploration rate ε_i	0.15

Table 6 Simulation Parameters for Scalability and Delay Performance Analysis

Setup	Parameter
Number of nodes	Variable (200—1000)
Node Speed	50 m/s
Nodes with Energy < 25%	10
Number of Packets Transmitted	2000 /s
Mobility Model	Steady State Random Way Point 3D
Wifi Standard	802.11ax
Area	1000 x 1000 x 1000 m
Communication range	250 m
Exploration rate ε_i	0.15

Fig. 9 Effect of Node Density on Delay and Distance with Fixed Exploration Rate



of the network topology. Although routing efficiency fluctuates slightly with node density, no severe degradation is observed. The exploration rate of 0.15 proves effective in ensuring scalability and consistent performance across varying network conditions.

4.5 Comparative performance evaluation of routing protocols

4.5.1 Effect of node mobility on end-to-end delay performance

The simulation setup for analyzing the impact of node mobility on routing performance is shown in Table 7.

More frequent topology changes occur with increased mobility, and their impact is clearly observed in the performance of the routing protocols compared in the Fig. 6. The proposed method, **Energy-Aware DBSCAN and Mobility-Aware Balanced Q-Learning Opportunistic Routing Protocol (EDMBOPR)**, achieves the lowest average end-to-end

delay across various mobility speed variations, with an average delay of approximately **0.06 seconds**. In comparison, the average delays for the other protocols are higher: **RORQ** exhibits an average delay of around **0.13 seconds**, **DA-ETR** shows an average delay of about **0.09 seconds**, and **ESQAODV** has an average delay of approximately **0.17 seconds**. This improvement in EDMBOPR's performance is attributed to its ability to selectively exclude high-mobility-prone nodes during route exploration, utilizing an optimal exploration set based on the analysis shown in Fig. 6. The proposed routing protocol demonstrates superior adaptability to dynamic network conditions. In contrast, other protocols experience significant delays due to the absence of mobility-aware mechanisms in their design (Fig. 10).

4.5.2 Impact of mobility on packet delivery ratio (PDR)

The simulation parameters relevant to this analysis are presented in Table 7.

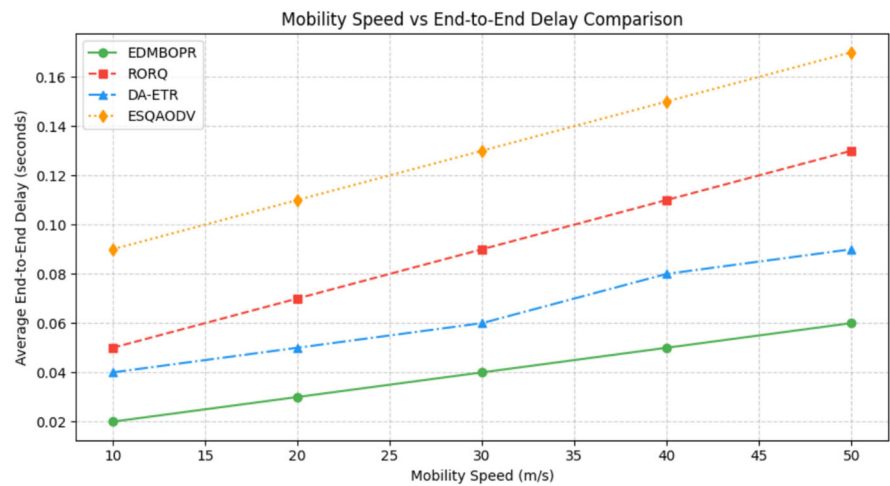
A general trend observed across all protocols is that increased node mobility leads to a reduction in the Packet Delivery Ratio (PDR). The proposed protocol, EDMBOPR, demonstrates strong performance even in highly dynamic mobility scenarios, achieving a PDR of **87%**, as illustrated in Fig. 11. In contrast, the RORQ protocol achieves only 75% PDR, while DA-ETR and ESQAODV drop to 0.79 and 0.81, respectively.

The superior performance of EDMBOPR can be attributed to its agent-based strategy. In dynamic environments, node agents proactively avoid selecting link-breakage-prone nodes during the exploration phase, using mobility information shared by neighboring nodes. In contrast, other protocols are more susceptible to link failures caused by the inclusion of

Table 7 Simulation parameters for mobility impact analysis

Setup	Parameter
Number of nodes	100
Node Speed	Variable (10—50 m/s)
Nodes with Energy < 25%	10
Number of Packets Transmitted	2000 /s
Mobility Model	Steady State Random Way Point 3D
Wifi Standard	802.11ax
Area	1000 x 1000 x 1000 m
Communication range	250 m
Exploration rate ε_i	0.15

Fig. 10 Comparison of End-to-End Delay under Varying Node Mobility Speeds for Different Routing Protocols



highly mobile nodes, leading to lower PDR values than those achieved by EDMBOPR.

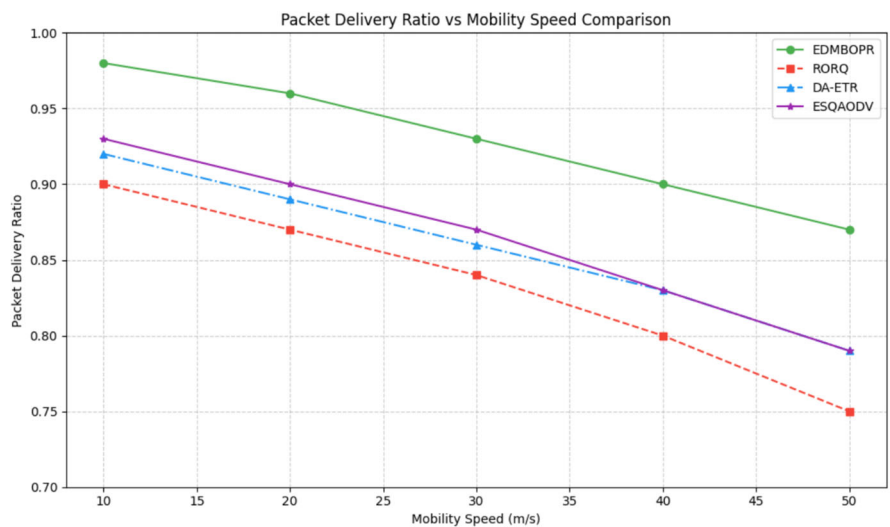
5 Discussion and conclusion

The primary focus of the proposed EDMBOPR method is the optimization of the agent's decision-making process. In highly dynamic environments, issues such as reduced packet delivery ratio (PDR) and increased delay are common. However, the proposed method achieves a PDR of **87%** across various mobility speed scenarios, outperforming existing protocols like RORQ, DA-ETR, and ESQAODV. While the average end-to-end delay increases with mobility speed, it remains lower than that of the compared protocols. Specifically, the proposed method exhibits a linear increase in delay up to **0.06** seconds, which is significantly lower than the delay observed in other protocols under similar conditions.

To enhance network lifetime, the proposed method filters out low-energy nodes. While this approach improves the overall lifetime of the network, it introduces a trade-off by excluding certain nodes from the Intermediate First Forwarder List, potentially resulting in reduced Packet Delivery Ratio (PDR) and increased delay. This limitation in the current implementation could be addressed in future work by exploring adaptive energy filtration techniques, which would help optimize both packet delivery and latency. The DBSCAN algorithm is utilized to dynamically update the First Forwarder List, ensuring that neighboring nodes with insufficient energy or those beyond communication range are filtered out. A single-cluster approach based on a Euclidean distance threshold of 250 meters is employed, which simplifies the computation with a complexity of $O(1)$.

In addition to these network optimization aspects, the proposed method demonstrates high scalability. As node density increases, the method maintains a constant delay of 0.1 seconds, showcasing its ability to handle large-scale networks.

Fig. 11 Packet Delivery Ratio Comparison under Varying Mobility Speeds



However, a slight linear increase in delay occurs when node density increases from 500 to 800 nodes, as the agent's exploration search space expands.

Furthermore, the performance of the customized epsilon-greedy strategy is demonstrated through key metrics. The total number of actions performed by the agent is 25,687, with 13,048 exploration actions and 12,639 exploitation actions. The accumulated total reward was 185,628.90. The ratio of exploration to exploitation is approximately **1.033**, which indicates a balanced approach that enhances the agent's decision-making. This metric suggests that the strategy effectively balances exploration and exploitation in dynamic environments. Notably, the actual exploration-exploitation actions indicate that the strategy placed a stronger emphasis on exploration (13,048) than on exploitation (12,639), highlighting its adaptability in exploring new actions beyond initial expectations.

6 Future work

To enhance the security of mobility information sharing among nodes, future work will focus on integrating blockchain technology as a robust and reliable security mechanism. Specifically, a permissioned blockchain architecture with a Proof of Authority (PoA) consensus mechanism will be implemented, tailored for Internet of Medical Things (IoMT) environments. Similar approaches have been successfully demonstrated in prior research [24, 25], highlighting the effectiveness of blockchain in securing communication within IoT networks.

The proposed integration will secure mobility data by storing encrypted blocks containing node locations, velocities, and trajectory information. These blocks will be protected using digital signatures and smart contracts [24], ensuring both data integrity and authenticity. The blockchain-based system will enable tamper-proof mobility records, authenticated movement updates, and secure handover processes, while also preserving location privacy, as discussed in [24].

Overall, this enhancement aims to mitigate existing vulnerabilities in mobility information sharing by preventing unauthorized access, data manipulation, and a range of potential security threats.

Another approach to securing mobility data is to avoid sharing mobility information altogether. Instead, machine learning models can be used to predict mobility patterns, reducing the need for explicit mobility sharing between nodes [10]. This would eliminate the overhead introduced by mobility data sharing, thereby mitigating security risks related to exposing mobility patterns. However, this approach also faces challenges such as the risk of overfitting and limited generalization, especially in the dynamic and unpredictable environments of MANETs. Careful consideration is needed

to balance model complexity and adaptability to avoid these issues while ensuring effective routing and maintaining security.

Despite these limitations, the proposed method achieves a robust Packet Delivery Ratio (PDR) of 87%, underscoring the effectiveness of optimized agent exploration in improving routing performance. As part of future work, efforts will be directed toward further refining the exploration—exploitation balance and minimizing mobility-induced overhead to enhance overall network efficiency and security.

Author Contributions A.D.E. conceptualized the study, proposed the use of reinforcement learning techniques in the design of opportunistic routing protocols in MANET, and introduced the concept of utilizing mobility information shared by neighboring nodes. D.G.D.A. identified and addressed challenges in reinforcement learning, particularly the exploration-exploitation dilemma, and focused on filtering low-energy nodes in the initial candidate forwarder selection process using DBSCAN. D.G.D.A. also introduced the concept of threshold distance calculation based on neighbor mobility information and distance to the neighbor, created the simulation environment, and contributed to the analysis. A.D.E. provided plotting support and proofreading for the manuscript. Both A.D.E. and D.G.D.A. reviewed and approved the final manuscript.

Funding This research received no external funding.

Data Availability The research data supporting the findings of this study is based on a simulation setup, and the algorithm steps are described in detail within the manuscript. These steps allow any user to generate the same simulation scenario and obtain comparable results, such as the average end-to-end delay. However, the raw data itself is not publicly available as it can be recreated using the described methodology.

Declarations

Competing interests The authors declare no competing interests.

References

1. Biswas S, Morris R (2005) Exor: Opportunistic multi-hop routing for wireless networks. In: Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '05), pp 133–144. Association for Computing Machinery, New York, NY, USA . <https://doi.org/10.1145/1080091.1080108>
2. Ryu J, Kim S (2023) Reputation-based opportunistic routing protocol using q-learning for manet attacked by malicious nodes. IEEE Access 11:47701–47711. <https://doi.org/10.1109/ACCESS.2023.3279177>
3. Hu T, Fei Y (2010) Qelar: A machine-learning-based adaptive routing protocol for energy-efficient and lifetime-extended underwater sensor networks. IEEE Trans Mob Comput 9(6):796–809. <https://doi.org/10.1109/TMC.2010.52>
4. Thorat SA, Kulkarni PJ (2015) Opportunistic routing in presence of selfish nodes for manet. Wireless Pers Commun 82(2):689–708. <https://doi.org/10.1007/s11277-014-2247-4>
5. Alsaqour RA, Abdelhaq MS, Alsukour OA (2012) Effect of network parameters on neighbor wireless link breaks in gpsr protocol and enhancement using mobility prediction model. EURASIP

- J Wireless Commun Netw 2012:1–15. <https://doi.org/10.1186/1687-1499-2012-171>
6. Moghaddam NA, Maleki A (2019) Rf energy harvesting system and circuits for charging of wireless devices using spectrum sensing. In: Amin M, Zhang YD (eds) Spectrum Sensing. IntechOpen, London. Chap. 3. <https://doi.org/10.5772/intechopen.84895>. <https://www.intechopen.com/chapters/65618>
7. Tavakoli HR, Aghaei M, Dadras H, Mahaki B (2019) The effect of battery charge levels of mobile phone on the amount of electromagnetic waves emission. J Biomed Phys Eng 9(5):545–552
8. Hong S-Y, Lee S-H, Pyun J-Y (2016) Rssi-based distance estimation framework using a kalman filter for sustainable indoor computing environments. Sustainability 8(11):1136. <https://doi.org/10.3390/su8111136>
9. Yang Z, Liu Y (2023) Distance measurement based on rssi and shadowing model in wireless sensor networks. In: AIP Conference Proceedings, vol. 2804, p. 040013. AIP Publishing, Melville, NY. <https://doi.org/10.1063/5.0137391>
10. Zhao B, Zhang J, Chen M, Liu X et al (2023) Fl-amm: Federated learning augmented map matching with heterogeneous cellular moving trajectories. IEEE J Sel Areas Commun 41(12):3494–3507. <https://doi.org/10.1109/JSAC.2023.3322841>
11. Alshamrani A, Shen X, Liu L (2014) Reinforcement learning for routing in cognitive radio ad hoc networks. EURASIP J Wirel Commun Netw 2014(1):1–13. <https://doi.org/10.1186/1687-1499-2014-179>
12. IEEE Computer Society (2021) IEEE Standard for Information technology–Telecommunications and information exchange between systems Local and metropolitan area networks–Specific requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications–Amendment 5: Enhancements for High Efficiency WLAN. IEEE Std 802.11ax-2021 (Amendment to IEEE Std 802.11-2020). Available at: <https://ieeexplore.ieee.org/document/9442429>. <https://doi.org/10.1109/IEEESTD.2021.9442429>
13. Singaravelan M, Thangam S et al (2024) Improving quality of service using an innovative energy effective reinforcement based path finding in mobile ad hoc network. In: Proceedings of the 2024 2nd International Conference on Computing and Data Analytics (ICDDA), pp 1–4. <https://doi.org/10.1109/ICDDA64887.2024.10867371>. IEEE
14. Pushpalatha K, Sherubha P, Sasirekha SP, Anguraj DK (2024) A constructive delay-aware model for opportunistic routing protocol in manet. Expert Syst Appl 124527. <https://doi.org/10.1016/j.eswa.2024.124527>
15. Yang T, Tang H, Liu J, Hao J, Meng Z (2023) Exploration in deep reinforcement learning: From single-agent to multiagent domain. IEEE Trans Neural Netw Learn Syst 34(1):1–21. <https://doi.org/10.1109/TNNLS.2023.3236361>
16. Buttyán L, Hubaux J-P (2003) Stimulating cooperation in self-organizing mobile ad hoc networks. Mobile Netw Appl 8:579–592. <https://doi.org/10.1023/A:1025146013151>
17. Sharma S, Ghosh RK, Sangaiah AK (2021) An iot based uav network for military applications. In: 2021 International Conference on Computer Communication and Informatics (ICCCI), pp 1–6. IEEE, ????. <https://doi.org/10.1109/ICCCI50826.2021.9402466>
18. Hosseinzadeh M, Tanveer J, Rahmani AM, Aurangzeb K, Yousefpoor E, Yousefpoor MS, Darwesh A, Lee S-W, Fazlali M (2024) A q-learning-based smart clustering routing method in flying ad hoc networks. J King Saud Univ-Comp Inf Sci 36(1):101894. <https://doi.org/10.1016/j.jksuci.2022.101894>
19. Duan S, Lyu F, Wu H, Chen W, Lu H, Dong Z, Shen X (2022) Moto: Mobility-aware online task offloading with adaptive load balancing in small-cell mec. IEEE Trans Mob Comput 23(1):645–659. <https://doi.org/10.1109/TMC.2021.3090706>
20. Das ADG, Devi EA (2024) Impact of node mobility in manet & analysis of bonnmotion mobility models using ns-3. In: 2024 1st International Conference on Advances in Computing, Communication and Networking (ICAC2N), pp 1466–1473. IEEE, Noida, India. <https://doi.org/10.1109/ICAC2N58929.2024.10895122>. <https://ieeexplore.ieee.org/document/10895122>
21. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E (2011) Scikit-learn: Machine learning in python. J Mach Learn Res 12:2825–2830
22. Hunter JD (2007) Matplotlib: A 2d graphics environment. Comput Sci Eng 9(3):90–95. <https://doi.org/10.1109/MCSE.2007.55>
23. Reback J et al. Pandas-dev/pandas: Pandas. <https://doi.org/10.5281/zenodo.3509134>
24. Malik A, Tyagi V, Singh S, Kumar R, Wu H, Gill SS (2024) Optimizing secure data transmission in 6g-enabled iomt using blockchain integration. IEEE Trans Consum Electron. <https://doi.org/10.1109/TCE.2024.3510812>
25. Liu W, Zhang M, Chen H (2024) Blockchain-based security enhancement for IoMT networks: Challenges and solutions. IEEE Trans Netw Serv Manage 21(1):218–233. <https://doi.org/10.1109/TNSM.2023.3287456>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Mr. A. Daniel Gurwin Das is currently pursuing a Ph.D. in Electronics Engineering at Sathyabama Institute of Science and Technology, Chennai, India. He is also an **Assistant Professor** in the Department of Computing at Kings Cornerstone International College. He holds a Master's degree in Communication Systems and a Bachelor's degree in Electronics and Communication Engineering. His research interests include **MANETs, Opportunistic routing, Reinforcement learning, Deep Double Q Network, Network Security and Wireless communication**. He has published in international conferences and journals and has over 8 years of teaching experience. Additionally, he is a certified CCNA professional, showcasing his expertise in networking.

ing, Reinforcement learning, Deep Double Q Network, Network Security and Wireless communication. He has published in international conferences and journals and has over 8 years of teaching experience. Additionally, he is a certified CCNA professional, showcasing his expertise in networking.



Dr. E. Anna Devi received her Ph.D. from Sathyabama Institute of Science and Technology, Chennai, in 2019. She holds a Master's degree in Communication and Networks (M.E.), which she completed in 2009 at the prestigious Madras Institute of Technology, Anna University, Tamil Nadu. Currently serving as an **Associate Professor** in the Department of Electronics and Communication Engineering at Sathyabama Institute of Science and Technology, she brings over **15 years of rich**

academic experience to her role. Her primary research interests span **Wireless Sensor Networks, Network Security, Deep Learning, and Image Processing**. A dedicated mentor, she has guided numerous students through their academic and project pursuits. Dr. Anna Devi is a prolific researcher, having authored and published **over 50 peer-reviewed research articles** in reputed international journals such as Elsevier and Springer. She has also actively contributed to global academic discourse by presenting her work at various international conferences. In addition to her research contributions, she has **filed four Indian patents** and has authored **more than five books** in the domain of Communication Engineering. She frequently participates in workshops and seminars, particularly in the area of Wireless Sensor Networks, staying abreast of emerging technologies and innovations. She is also a **life member of the Institution of Engineers (India) [IEI]**, reflecting her commitment to professional excellence and continuous learning.